

School of Future Tech

Case Study Report

on

Mental Health Case Tracking System

by

Ashish SK Gupta

150096725084

Index

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. Case Study Design
4. Methods & Technology Applied
5. Implementation Details
6. Results and Conclusion
7. References

1. Introduction to the Case Study

Mental health is one of the most sensitive areas of healthcare. Patients require long-term, confidential, and well-organised tracking of their treatment history. This case study presents a Mental Health Case Tracking System built entirely in C++ using only the standard library.

The system is designed at beginner-level CSE student difficulty — no external libraries, no database, and no complex frameworks. It demonstrates how fundamental C++ concepts such as file I/O, string parsing, functions, and control flow can be combined to build a practical application.

2. Problem Statement / Case Background (Abstract)

Background

Mental health counsellors currently manage patient records on paper or in unorganised files. This creates risks of data loss, privacy issues, and difficulty in tracking long-term treatment. A purpose-built digital system is needed.

Abstract

This case study presents a Mental Health Case Tracking System written in C++. The system provides user registration and login with credentials stored in `users.txt`, unique case registration with auto-generated IDs, session logging per case, and comma-separated flat-file storage across three `.txt` files. The entire solution is implemented in under 200 lines of beginner-level C++ using only standard library headers.

3. Case Study Design

The system is designed as a modular set of functions, each doing one specific job:

8. Authentication Module — Users register with a username and password. Credentials are saved in `users.txt`. On login, the program reads the file and checks if the username and password match.
9. Case Management Module — Cases are registered with auto-generated unique IDs (C0001, C0002...), patient initials, age, primary concern, severity, and status. Saved in `cases.txt`.
10. Session Tracking Module — Therapy sessions are logged with auto-generated session IDs (S0001...), linked to a case ID, with duration in minutes and brief notes. Saved in `sessions.txt`.
11. Storage Module — All data is saved in comma-separated `.txt` files. Data persists across program runs.
12. CLI Menu / Dashboard — A clean text menu connects all modules. The user navigates using number choices after login.

4. Methods & Technology Applied

Key Methods Used

- Comma-separated flat-file storage — each record is one line, fields separated by commas. Easy to read and write using standard C++ file operations.

- Auto-incrementing ID generation — the program counts existing lines in the file and adds 1 to create the next ID (C0001, C0002...).
- String parsing using `find()` and `substr()` — used to split a comma-separated line into individual fields like username, password, case ID etc.
- String splitting using `stringstream` and `getline()` — used in `showCases()` to break each line at commas and display fields in neat columns.
- In-place file rewriting for status updates — all lines are read into an array, the target line is modified, then all lines are written back.

Technology Stack

- Language: C++ (C++11 or later)
- Headers used: `iostream`, `fstream`, `sstream`, `string`, `io manip`
- Storage: comma-separated .txt files (`users.txt`, `cases.txt`, `sessions.txt`)
- Build command: `g++ mental_health_tracker.cpp -o tracker`
- Environment: Any terminal — Windows, Linux, or macOS

5. Implementation Details

The code is written in a single file with clearly named functions. Each function handles one specific task:

- `showHeading(title)` — prints a formatted banner heading on the terminal screen.
- `registerUser()` — asks for a username and password, checks if username already exists in `users.txt`, then saves the new credentials.
- `loginUser()` — reads `users.txt` line by line, splits each line at the comma, and compares the stored username and password with what the user typed. Returns the username if matched.
- `showCases()` — reads `cases.txt`, splits each line by comma using `stringstream`, and prints each field in a neat aligned table using `setw()`.
- `addCase()` — counts lines in `cases.txt` to generate the next case ID, asks for patient details, and saves everything as one comma-separated line.
- `updateStatus()` — reads all lines into an array of 100 strings, finds the matching case ID using `substr()`, replaces the last field (status) using `rfind(',')`, then writes all lines back.
- `addSession()` — checks if the case ID exists, generates a session ID, asks for notes and duration, and appends to `sessions.txt`.
- `viewSessions()` — reads `sessions.txt` and optionally filters lines by case ID using `string::find()`. Displays matching lines.
- `dashboard(username)` — a do-while loop showing the menu after login. Calls the appropriate function based on the user's number input.
- `main()` — entry point. Shows the main menu in a do-while loop. Handles login, register, and exit choices.

6. Results and Conclusion

Results

- User registration and login works correctly — credentials saved and verified from users.txt.
- Case registration produces unique IDs and displays all cases in a properly formatted table.
- Session tracking correctly links sessions to cases and saves duration and notes.
- All data persists across program restarts — stored in .txt files on disk.
- System compiles with zero errors using standard g++ on any platform.
- Entire implementation is under 200 lines of clean, readable, beginner-level C++ code.

Conclusion

This case study presents a complete Mental Health Case Tracking System that meets all stated objectives. Users can register and login securely. Cases and sessions are tracked with unique IDs and saved persistently. The code uses only basic C++ — file reading and writing, strings, loops, and functions — making it accessible to beginner CSE students while demonstrating a real, working application.

7. References

13. C++ Standard Library Reference — <https://en.cppreference.com/>
14. C++ fstream documentation — <https://cplusplus.com/reference/fstream/>
15. C++ string find and substr — <https://cplusplus.com/reference/string/string/find/>
16. ITM Skills University, School of Future Tech — Case Study 31 Problem Statement
17. C++ iomanip reference (setw) — <https://cplusplus.com/reference/iomanip/>